

PROGRAMMING FOR PROBLEM SOLVING (3110003)



Prepared By,

Prof. Shraddha Modi

Computer Engineering

LDCE, Ahmedabad

CHAPTER-2

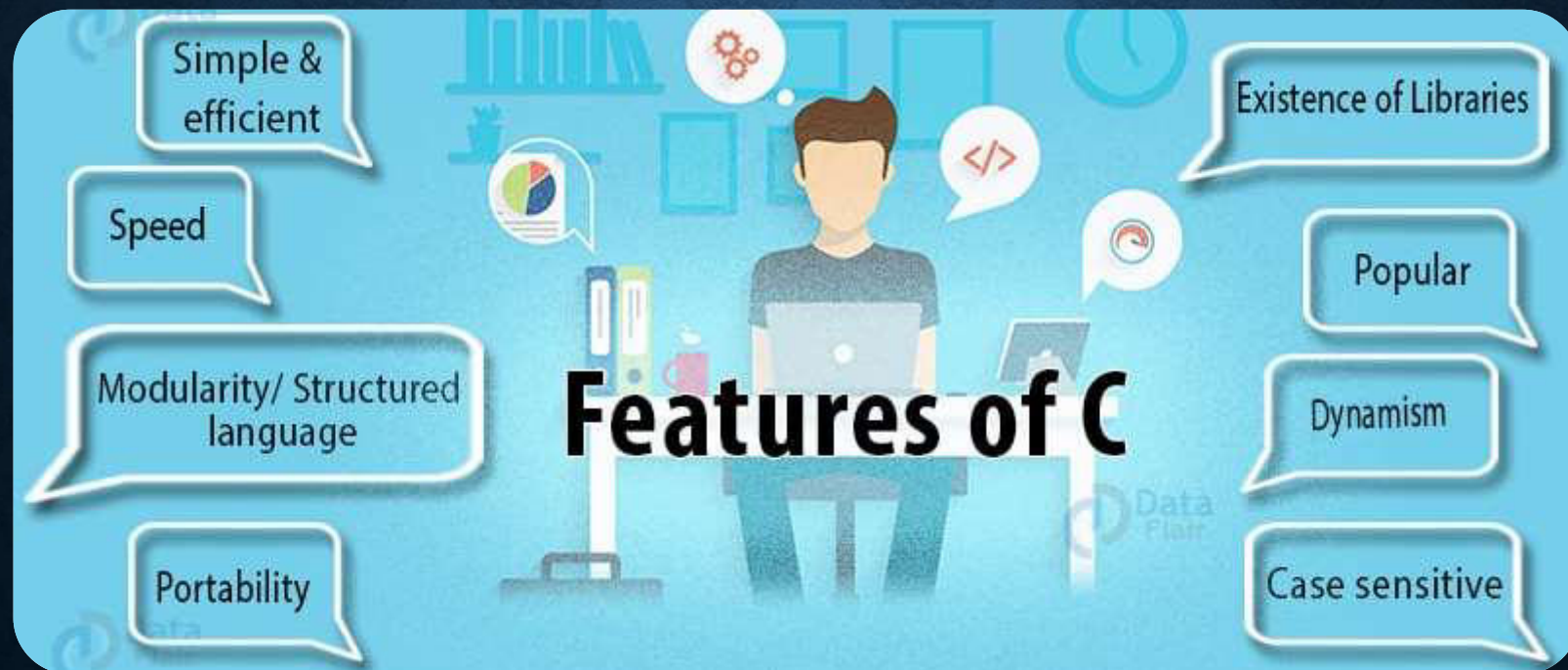
BASICS OF C PROGRAMMING

Prepared By,
Prof. Shraddha Modi
Computer Engineering
LDCE, Ahmedabad

ROADMAP

- ❑ Features of C language,
- ❑ Structure of C Program (comments, header files)
- ❑ Constants and variables
- ❑ Data types,
- ❑ Operators,
- ❑ Expressions, evaluation of expressions,
- ❑ Type conversion,
- ❑ Precedence and associativity,
- ❑ I/O function
- ❑ Preprocessor (Self study)

Features of C language



IMPORTANCE/FEATURES OF C

- Robust language → built in functions and operators.
- C has set of data types and operators so C is efficient and fast.
- C is highly portable.
- C has only 32 keywords which are easy to remember.
- C has set of built in functions and standard functions.
- C has system library. And We can add our own functions to C library it means C can be extended itself.

STRUCTURE OF C PROGRAM

```
//program to print area.  
#include<stdio.h>  
#define PI 3.1415  
float radius;  
float area();  
int main()  
{  
    float a;  
    printf("Enter radius : ");  
    scanf("%f",&radius);  
    a = area();  
    printf("Area of Circle : %f",a);  
}  
float area()  
{  
    return (PI * radius * radius);  
}
```

Documentation Section
Link Section
Definition Section
Global Declaration Section
Main Function Section void main()/int main() Local Declaration Part Executable Code Part
Sub Program Section Function1() Function2() FunctionN()

DOCUMENTATION SECTION

COMMENTS

- A comment is an explanation or description of the source code of the program.
- It helps a programmer to explain logic of the code and improves program readability.
- At run-time, a comment is ignored by the compiler.
- There are two types of comments in C:
 - Single line comment
 - Multi-line comment

DOCUMENTATION SECTION

COMMENTS

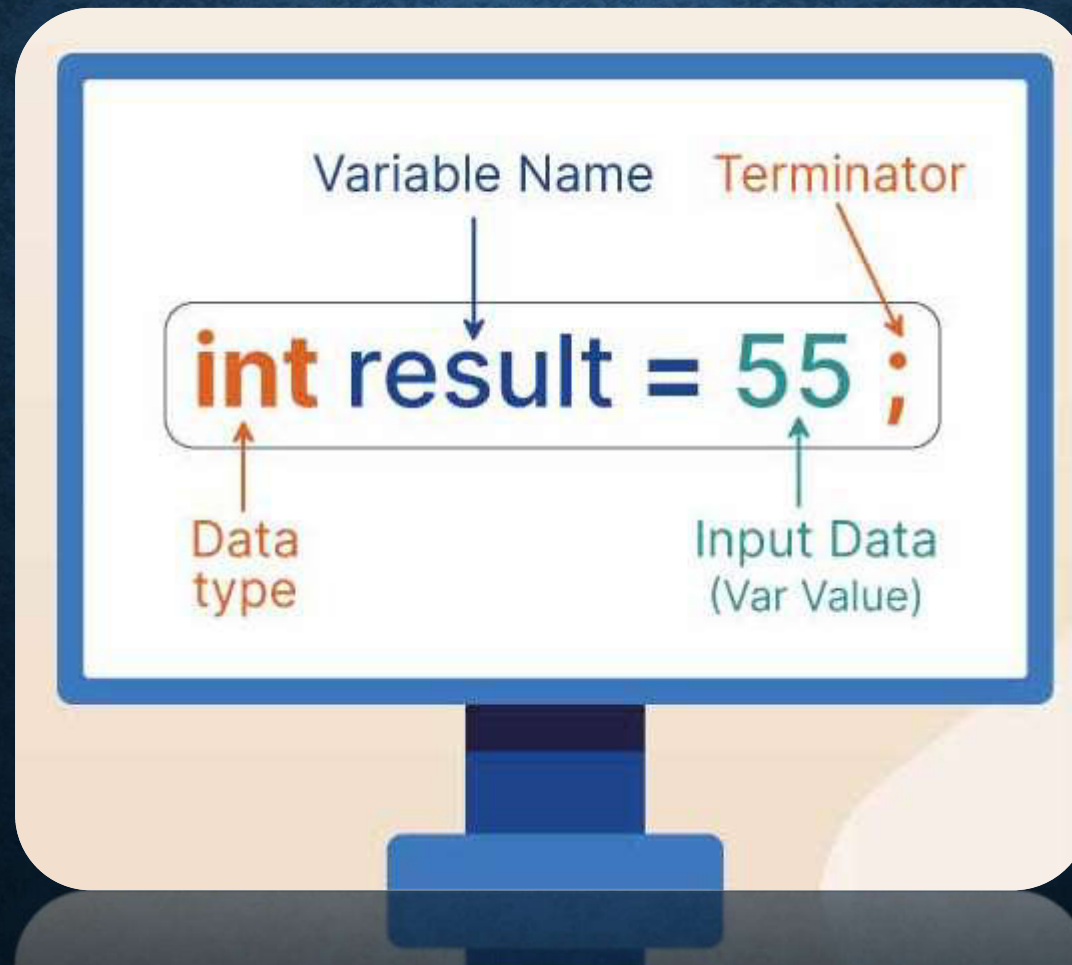
- There are two types of comments in C:
 - **Single line comment**
 - Represented as `//` double forward slash
 - It is used to denote a single line comment only.
 - Example: `// Single line comment`
 - **Multi-line comment**
 - Represented as `/* any_text */` start with forward slash and asterisk (`/*`) and end with asterisk and forward slash (`*/`).
 - It is used to denote single as well as multi-linecomment.
 - Example: `/* multi line comment line -1`
 - `multi line comment line -2 */`

LINK SECTION HEADER FILES

- A header file is a file with extension .h which contains the set of predefined standard library functions.
- The “#include” preprocessing directive is used to include the header files with extension in the program.

Header file	Description
stdio.h	Input/Output functions (printf and scanf)
conio.h	Console Input/Output functions (getch and clrscr)
math.h	Mathematics functions (pow, exp, sqrt etc...)
string.h	String functions (strlen, strcmp, strcat etc...)

Constants and Variables



VARIABLES AND CONSTANTS

□ Variable is a **symbolic name** given to some value which can be changed.

□ $x, y, a, \text{count}, \text{etc.}$ can be variable names.

□ $x=5 \quad a=b+c$

□ Constant is a fixed value which cannot be changed.

□ $5, -7.5, 1452, 0, 3.14, \text{etc.}$

CHARACTER SET

- Characters are used to write tokens. These characters are defined by the **Unicode character set**.
- The C character set includes letters, digits, special characters and white spaces as building blocks to form basic programming elements (C tokens).

C CHARACTER SET

Letters	Digits
Uppercase A.....Z Lowercase a.....z	Decimal digits 0.....9
Special Characters	White Spaces
! * + \ “ < # (= { > %) ~ ; } / ^ - [: , ? & _] . ‘	Back space \b Horizontal tab \t Carriage return \r New line \n Form feed \f

TOKENS

- Smallest individual units in a program are known as **tokens**. C has six types of tokens as given below:

- | | |
|--------------------|---------------------------|
| 1. Keywords | Ex: float, int, while, if |
| 2. Identifier | Ex: amount, sum, area |
| 3. Constants | Ex: -15.5, 100 |
| 4. String | Ex: “year”, ”hello” |
| 5. Special Symbols | Ex: [], { } |
| 6. Operators. | Ex : +, -, * |

KEYWORDS

- Predefined word is known as keywords.
- C language has 32 reserved keywords.
- Since keywords have specific meaning, we cannot use them as identifiers.
- All keywords are to be written in lower – case letters.

KEYWORDS

auto	extern	sizeof	break	float	static
case	for	struct	char	goto	switch
const	if	typedef	continue	int	union
default	long	unsigned	do	register	void
double	return	signed	else	short	while
enum	signed				

IDENTIFIER

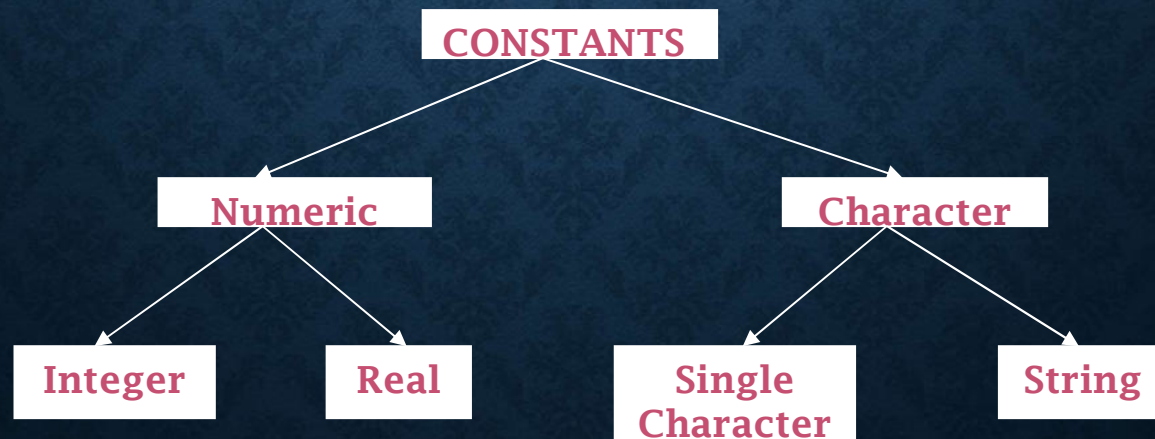
- They are used for naming variables, functions and arrays.

Identifiers follow the below listed rules:

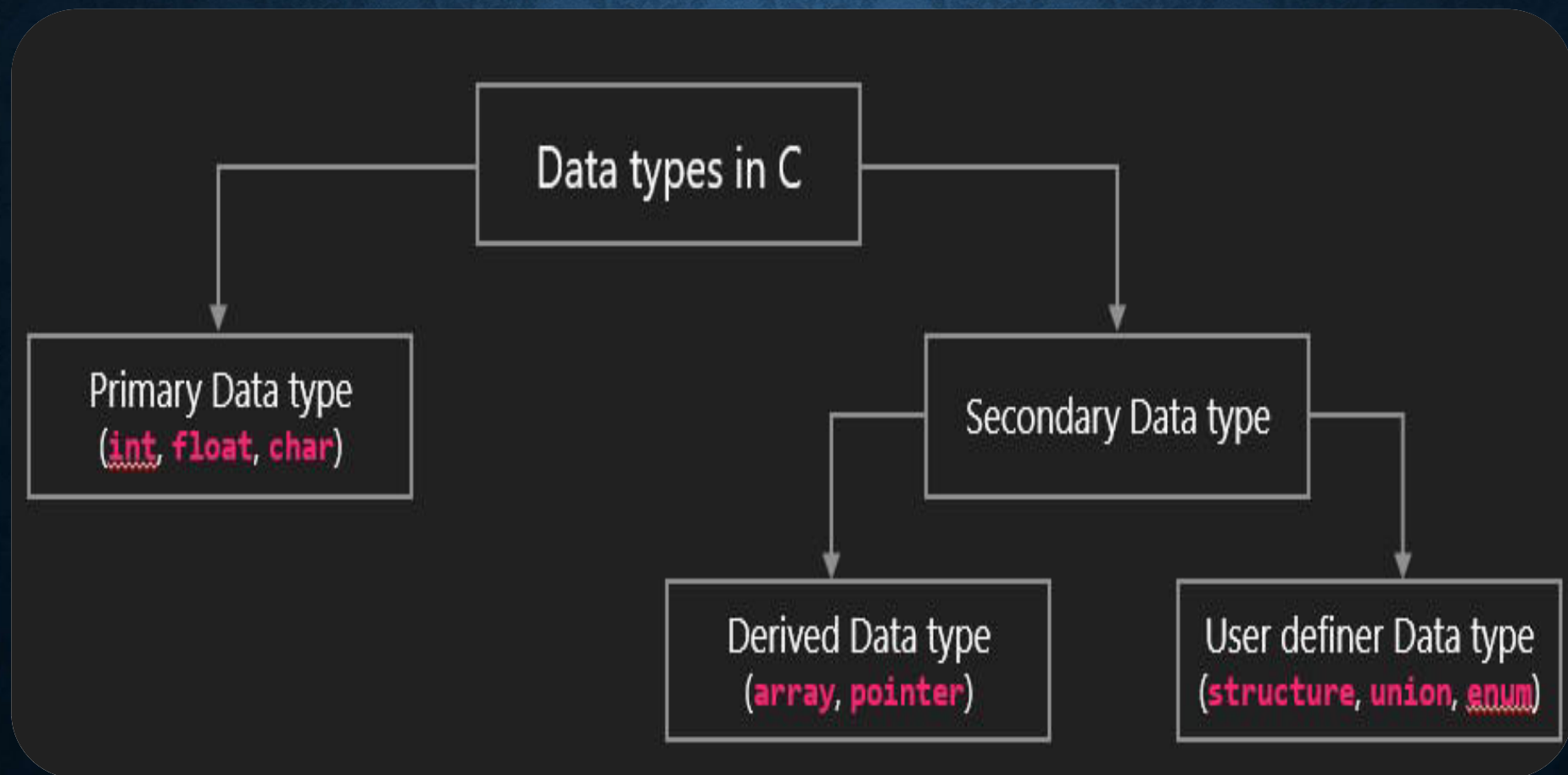
1. Alphabets, digits, underscores are permitted.
2. They must not begin with a digit.
3. First character must be a letter. They may also begin with an underscore.
4. Uppercase and lower case letters are distinct.
5. They can be of 31 characters length however the first 8 characters are treated as significant by the C compiler.
6. Declared keywords cannot be used as a variable.

CONSTANT

- Constant means fixed value.
- The value of the constant cannot change during execution of program.
- C supports following types of constant:



Datatype



DATA TYPES IN C

- A Data Type is a type of data.
- A data type is a data storage format that can contain a specific type or range of values.
- ANSI C Supports Three classes of data types.
 1. Primary data type(fundamental)
 2. Derived data types
 3. User defined data types

DATA TYPES IN C

- All “C” compiler supports 5 fundamental data types:
 1. Integer (int)
 2. Character (char)
 3. floating point (float)
 4. double-precision (double)
 5. void

SIZE AND RANGE OF DATA TYPE

Data type	Bits in RAM (1 byte = 8 bits)	Range of data type
<u>char</u> or signed char	8	-128 to 127
unsigned char	8	0 to 255
<u>int</u> or signed int	16	-32,768 to 32,767
unsigned int	16	0 to 65535
short int or signed short int	8	-128 to 127
unsigned short int	8	0 to 255
long int or signed long int	32	-2,147,483,648 to 2,147,483,647
unsigned long int	32	0 to 4,294,967,295
<u>float</u>	32	3.4E-38 to 3.4E+38 (6-digit precision)
<u>double</u>	64	1.7E-308 to 1.7E+308 (14-digit precision)
long double	80	3.4E-4932 to 1.1E+4932 (80-digit precision)

USER DEFINED DATA TYPES

- **Typedef : Syntax:** typedef type identifier

```
# include < stdio.h>
```

```
void main ( )
```

```
{
```

```
typedef int amt ;
```

```
amt Rupees = 20;
```

```
printf (" Rupees %d", Rupees);
```

```
}
```

Advantage: We can create meaningful data type names for increasing the readability of the program.

USER DEFINED DATA TYPES

- **enum: Syntax:** enum identifier {value1, value2, ... valuen};
- The idea is **that instead of using an int to represent a set of values**, a type with a restricted set of values is used instead.
- For example if we use the colors of the rainbow, which are Red, Orange, Yellow, Green, Blue, Indigo, Violet
- If enum didn't exist, you might use a **#define** to specify these values.
- For example: #define red 1
#define orange 2...

USER DEFINED DATA TYPES

Using Enum:

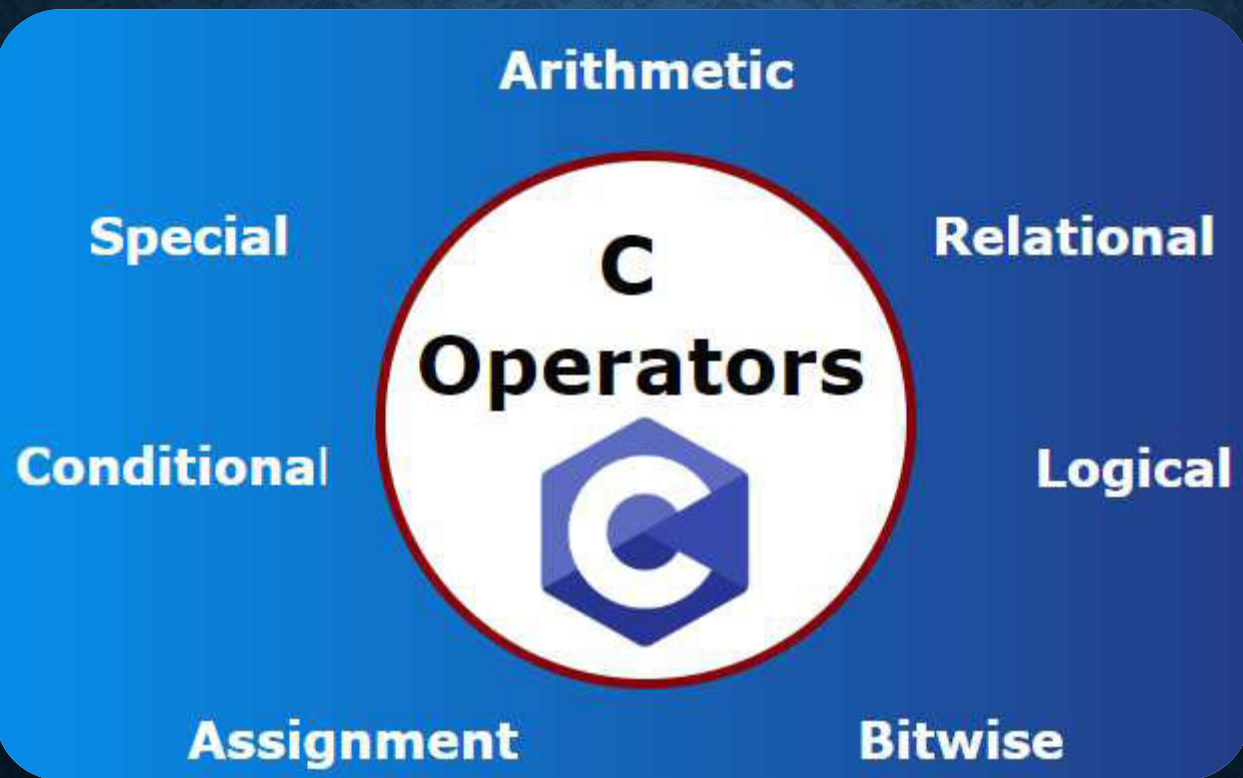
```
enum rainbowcolors { red, orange, yellow, green, blue,  
indigo, violet }
```

```
enum rainbowcolors trafficlights;
```

```
trafficlights= red;
```

- Now internally, the compiler will use an int to hold these and if no values are supplied, red will be 0, orange is 1 etc.

Operators



OPERATORS

- ❑ Operator: An operator is a symbol that tells the Computer to perform certain mathematical or logical manipulations.
- ❑ Expression: An expression is a sequence of operands and operators that reduces to single value.

OPERATORS

1. Arithmetic operators (+, -, *, /, %)
2. Relational operators (<, <=, >, >=, ==, !=)
3. Logical operators (&&, ||, !)
4. Assignment operators (+=, -=, *=, /=)
5. Increment and decrement operators (++ , --)
6. Conditional operators / Ternary operator(?:)
7. Bitwise operators (&, |, ^, <<, >>)
8. Special operators ()

ARITHMETIC OPERATORS (+, -, *, /, %)

Operator	Meaning	Example	Description
+	Addition	$a + b$	Addition of a and b
-	Subtraction	$a - b$	Subtraction of b from a
*	Multiplication	$a * b$	Multiplication of a and b
/	Division	a / b	Division of a by b
%	Modulo division- remainder	$a \% b$	Modulo of a by b

RELATIONAL OPERATORS (<, <=, >, >=, ==, !=)

- ❑ These are used to compare or determine relation among two operands.
- ❑ These are the operators used to Compare arithmetic, logical and character expressions.
- ❑ The value of a relational express is either one or zero.

Operator	Meaning	Example	Description
<	Is less than	$a < b$	a is less than b
<=	Is less than or equal to	$a <= b$	a is less than or equal to b
>	Is greater than	$a > b$	a is greater than b
>=	Is greater than or equal to	$a >= b$	a is greater than or equal to b
==	Is equal to	$a = b$	a is equal to b
!=	Is not equal to	$a != b$	a is not equal to b

LOGICAL OPERATORS (&&, ||, !)

- ❑ It is used to combine two or more relational expressions .
- ❑ Logical Operators are used when we want to test more than one condition and make decisions.

Operator	Meaning
&&	logical AND (Both non zero then true, either is zero then false)
	logical OR (Both zero then false, either is non zero then true)
!	Is greater than

a	b	a&& b	a b
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

LOGICAL OPERATORS (&&, ||, !)

Operator	Meaning
----------	---------

&&	AND
----	-----

a=5, b=7, c= 9

(a<b) && (a<c)

True(1)

(a>b) && (a<c)

False(0)

	OR
--	----

(b<a) || (a<c)

True(1)

(b<a) || (c<b)

False(0)

!	NOT
---	-----

!(a<b)

False(0)

!(a>b)

True(1)

ASSIGNMENT OPERATORS (+=, -=, *=, /=)



Operator	Meaning
=	Assigns value of right side to left side
+=	$a += 1$ is same as $a = a + 1$
-=	$a -= 1$ is same as $a = a - 1$
*=	$a *= 1$ is same as $a = a * 1$
/=	$a /= 1$ is same as $a = a / 1$
%=	$a \% = 1$ is same as $a = a \% 1$

ASSIGNMENT OPERATORS (+=, -=, *=, /=)

Operator	Symbol	Example	Result
Assignment Operator	=	a = 10;	
Addition Assignment Operator	+=	a+=15;	a=a+15
Subtraction Assignment Operator	-=	a-=10;	a=a-10
Multiplication Assignment Operator	*=	a*=n+1	a=a*(n+1)
Division Assignment Operator	/=	a/=n+1	a=a/(n+1)
Modulus Assignment Operator	%=	a%=2;	a=a%2

INCREMENT AND DECREMENT OPERATORS

(++, --)

- ❑ It is unary operator.
- ❑ The operation performed is to add one to or subtract one from the operand.

❑ Operator

Increment Operator

(++)

Decrement Operator

(--)

INCREMENT AND DECREMENT OPERATORS (++, --)

□ **++X OR X ++**

X+=1

X=X+1

□ **-- X OR X --**

X-=1

X=X-1

□ A *Prefix/pre-increment* operator first increment the operand and then the result is assigned to the variable on left.

□ A *postfix operator* first assigns the value to the variable on the left and the increments the operand.

m = 5;	m = 5;
y = ++m;	y = m++
O/P m =6, y=6	O/P m=6, y=5

INCREMENT AND DECREMENT OPERATORS (++, --)

- Show the output of below code:

```
void main ( )  
{  
    int x = 10, y = 20, z, a ;  
    z = x * y ++;  
    a = x * y ;  
    printf(" %d %d\n", z, a);  
    z = x * ++y;  
    a = x * y;  
    printf(" %d %d\n", z, a);  
}
```

OUTPUT:

200 210

220 220

INCREMENT AND DECREMENT OPERATORS (++, --)

- Show the output of below code:

```
void main()
{
    int count = 0, loop;
    loop = ++count;

    printf("loop = %d, count = %d\n", loop, count);

    loop = count++;
    printf("loop = %d, count = %d\n", loop, count);
}
```

Output:

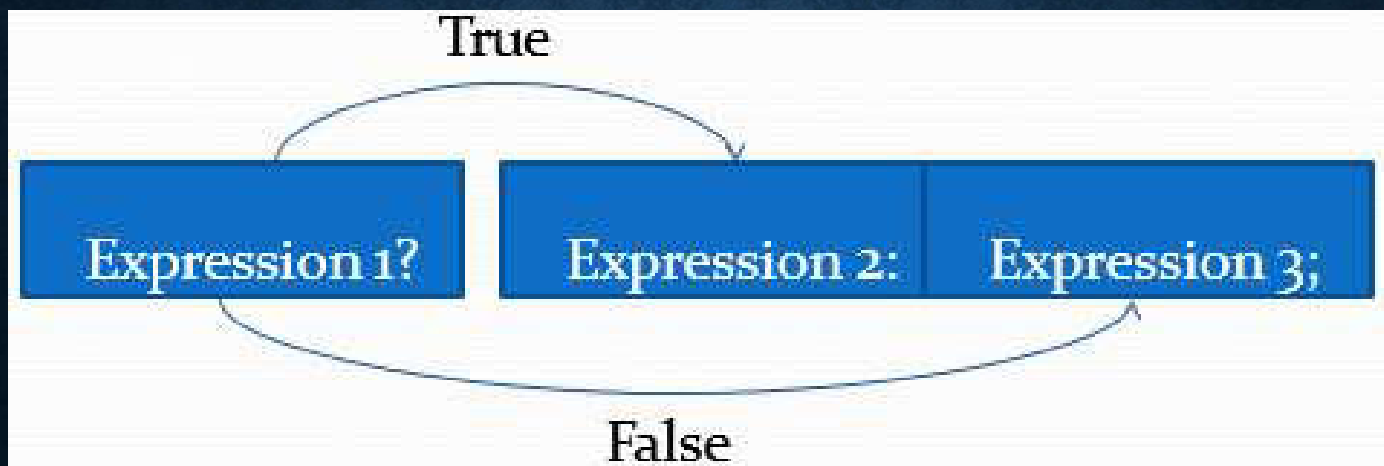
loop = 1, count = 1

loop = 1; count = 2

CONDITIONAL OPERATORS / TERNARY OPERATOR (?:)

- ❑ The Conditional operator is used to check condition whether true or false. The result will be as per calculated by the statement.
- ❑ Syntax for conditional operators is

Expression1 ? Expression 2 : Expression 3;



CONDITIONAL OPERATORS / TERNARY OPERATOR (?:)

Show the output of below code:

```
void main()
{
    int a,b;
    printf("\n\nEnter First No. = ");
    scanf("%d",&a);

    printf("\nEnter Second No. = ");
    scanf("%d",&b);

    (a<b) ? printf("First number is smaller")
          : printf("Second number is smaller");

    getch();
}
```


BITWISE OPERATORS (&, |, ^, <<, >>)



Operator	Meaning
&	bitwise AND
	bitwise OR
^	bitwise exclusive OR
<<	shift left
>>	shift right
~	Compliment Operator

Bit1	Bit2	Bit1&Bit2	Bit1 Bit2	Bit1 ^ Bit2
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

BITWISE OPERATORS (&, |, ^, <<, >>)

Show the output of below code:

```
void main()
{
int a = 12, b = 25;
printf("Output = %d", a & b);
}
```

```
void main()
{
int a = 12, b = 25;
printf("Output = %d", a | b);
}
```

```
12 = 00001100 (In Binary)
25 = 00011001 (In Binary)
```

Bit Operation of 12 and 25

```
  00001100
& 00011001
  _____
  00001000 = 8 (In decimal)
```

```
12 = 00001100 (In Binary)
25 = 00011001 (In Binary)
```

Bitwise OR Operation of 12 and 25

```
  00001100
| 00011001
  _____
  00011101 = 29 (In decimal)
```


BITWISE OPERATORS (&, |, ^, <<, >>)

Show the output of below code:

```
void main()
{
int a = 12, b = 25;
printf("Output = %d", a ^ b);
}
```

12 = 00001100 (In Binary)

25 = 00011001 (In Binary)

Bitwise XOR Operation of 12 and 25

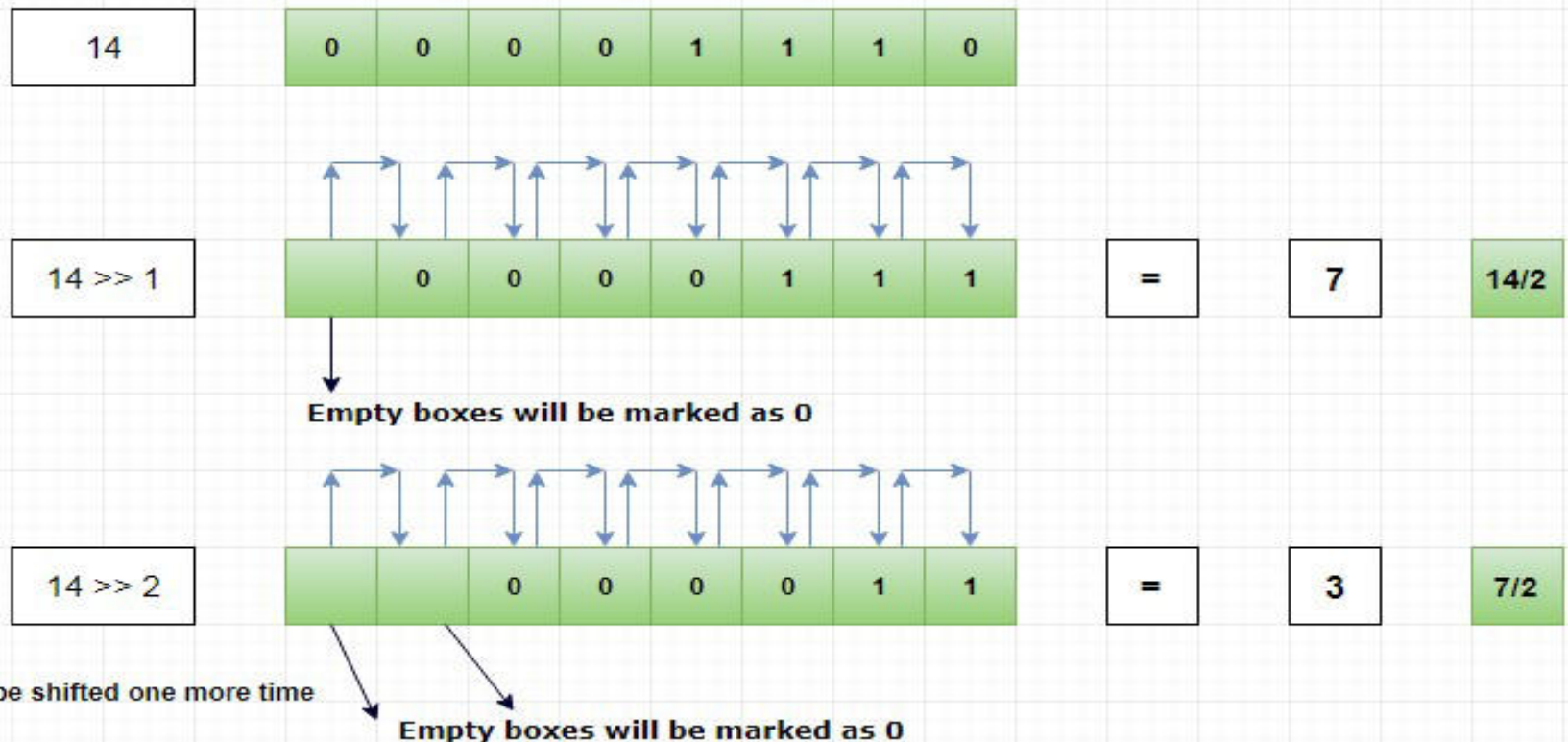
00001100

^ 00011001

00010101 = 21 (In decimal)

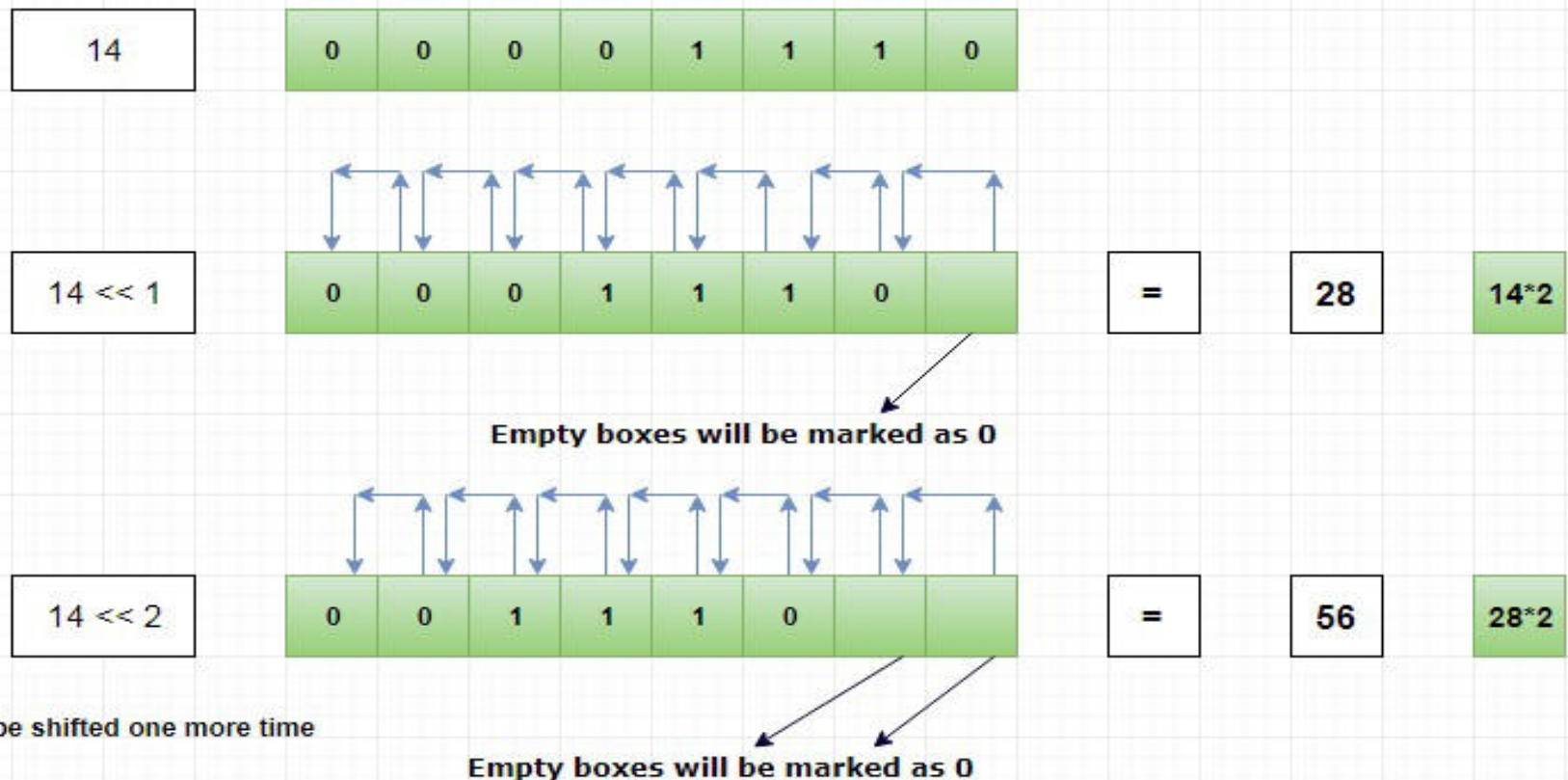
BITWISE OPERATORS (&, |, ^, <<, >>)

- ❑ Right Shift Operator - Shift right means divide by 2.
- ❑ Right shift operator shifts all bits towards right by certain number of specified bits. It is denoted by >>.



BITWISE OPERATORS (&, |, ^, <<, >>)

- ❑ Left Shift Operator - Shift left means multiply by 2
- ❑ Left shift operator shifts all bits towards left by a certain number of specified bits.



SPECIAL OPERATOR

Operator	Meaning
&	Address operator, it is used to determine address of the variable.
*	Pointer operator, it is used to declare pointer variable and to get value from it.
,	Comma operator. It is used to link the related expressions together.
<u>sizeof</u>	It returns the number of bytes the operand occupies.
.	member selection operator, used in structure.
->	member selection operator, used in pointer to structure.

EXPRESSIONS AND EVALUATION ON EXPRESSION

- An expression is evaluated based on the operator precedence and associativity.
- When there are multiple operators in an expression, they are evaluated according to their precedence and associativity.

EXPRESSIONS AND EVALUATION ON EXPRESSION

- **Operator precedence:** It dictates the order of evaluation of operators in an expression.
- **Associativity:** It defines the order in which operators of the **same precedence** are evaluated in an expression. Associativity can be either from left to right or right to left.

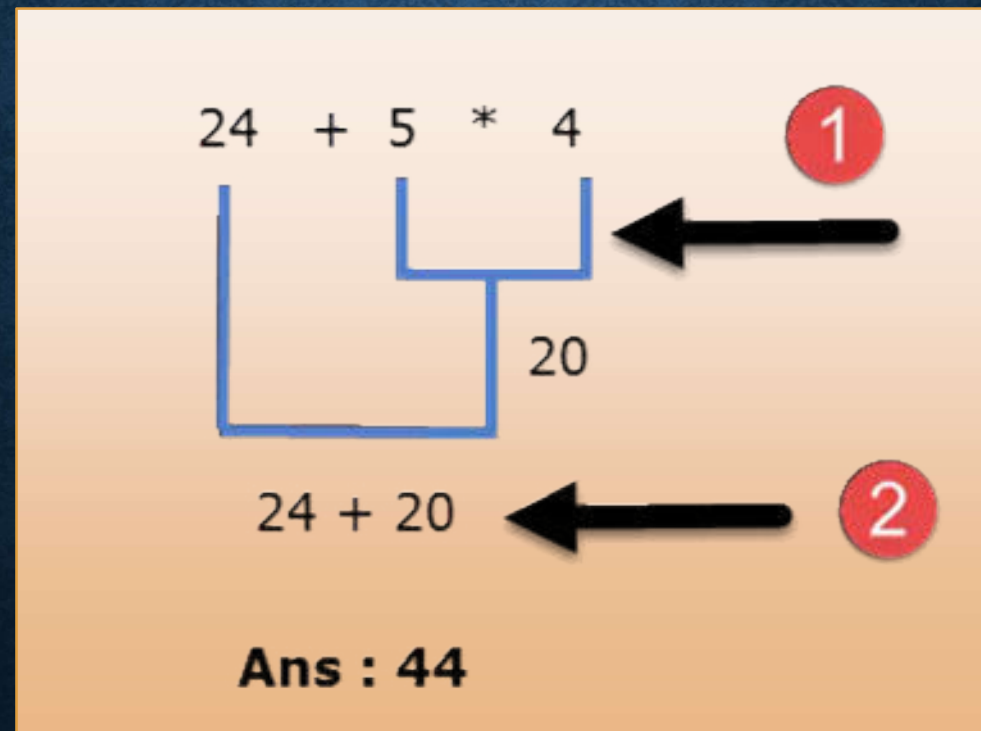
EXPRESSIONS AND EVALUATION ON EXPRESSION

Precedence	Operator	Associativity
1	() [] -> .	Left to right
2	++ --	Left to right
3	++ -- + - !~ (type) * & sizeof	Right to left
4	* / %	Left to right
5	+ -	Left to right
6	<< >>	Left to right
7	<<= >>=	Left to right
8	== !=	Left to right
9	&	Left to right
10	^	Left to right
11		Left to right
12	&&	Left to right
13		Left to right
14	?:	Right to left
15	= + = - = * = / = %= <<= >>= &= ^= =	Right to left

EXPRESSIONS AND EVALUATION ON EXPRESSION

□ Consider the following example:

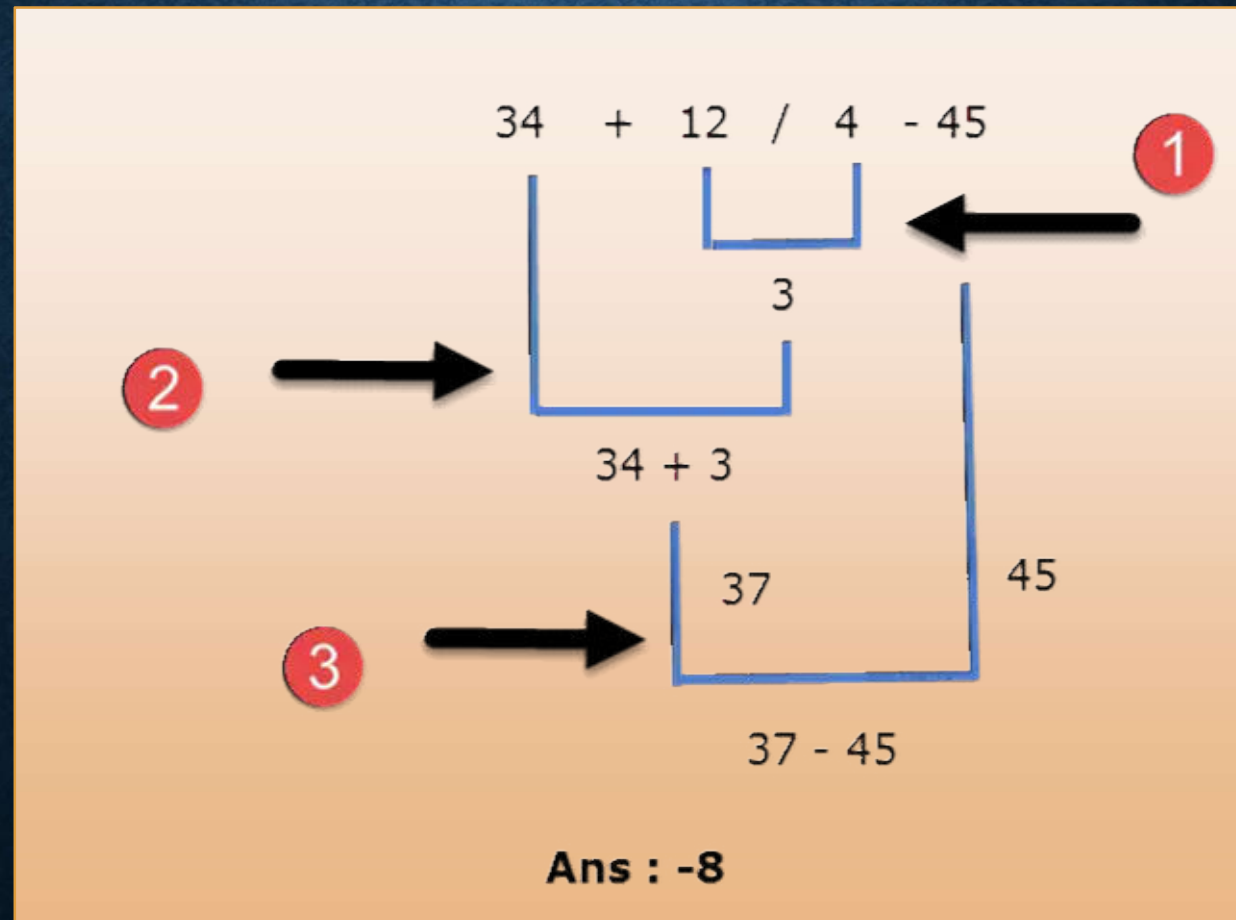
$$24 + 5 * 4$$



EXPRESSIONS AND EVALUATION ON EXPRESSION

□ Consider the following example:

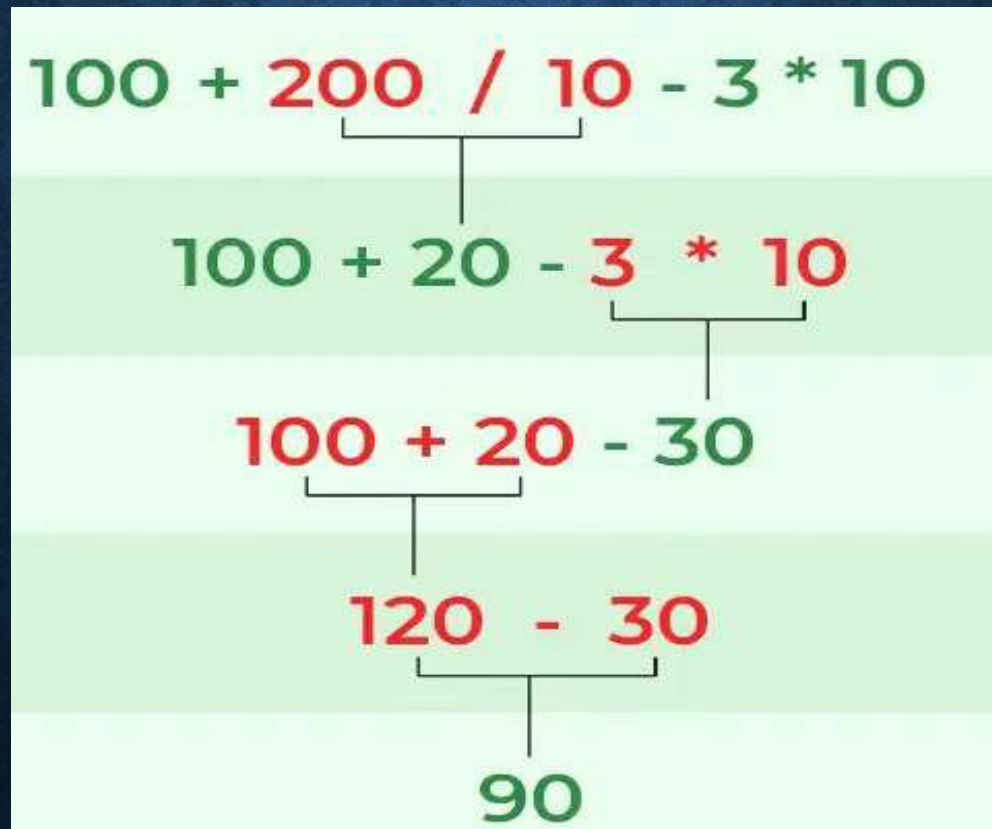
$$34 + 12/4 - 45$$



EXPRESSIONS AND EVALUATION ON EXPRESSION

□ Consider the following example:

$$\text{exp} = 100 + 200 / 10 - 3 * 10$$



EXPRESSIONS AND EVALUATION ON EXPRESSION

□ Consider the following example:

```
void main()
```

```
{
```

A. 5

```
int x = 30, y = 12;
```

B. 10

```
int z;
```

C. 15

```
z = x * 2 / y;
```

```
printf("\n output = %d", z);
```

D. 60

```
}
```

EXPRESSIONS AND EVALUATION ON EXPRESSION

□ Consider the following example:

```
void main()
```

```
{
```

A. 10

```
int x = 80, y = 3;
```

B. 30

```
int z;
```

C. 55

```
z = x / 2 * y;
```

```
printf("\n output = %d", z);
```

D. 120

```
}
```


SUMMARY

EXPRESSIONS AND EVALUATION ON EXPRESSION

1. Associativity is only used when there are two or more operators of the same precedence.
2. Parenthesis will change the order of evaluation.

Example: Consider the given expression

- $100 + 200 / 10 - 3 * 10 = 90$

But if we enclose $100 + 200$ in parenthesis, then the result will be different.

- $(100 + 200) / 10 - 3 * 10 = 0$

As the $+$ operator will be evaluated before $/$ operator.

SUMMARY

EXPRESSIONS AND EVALUATION ON EXPRESSION

3. Precedence and associativity of postfix ++ and prefix ++ are different.

The precedence of postfix ++ is more than prefix ++, their associativity is also different. The associativity of postfix ++ is left to right and the associativity of prefix ++ is right to left.

TYPE CONVERSION AND TYPE CASTING

- ❑ Type conversion : Automatic (lower type to higher type)/Implicit Type Conversion

- ❑ Rules:

- ❑ If float is assigned to integer, fractional part is truncated.
- ❑ If double is assigned to float, then rounding take place.
- ❑ If long int is assigned to integer, then additional bits are dropped.

- ❑ Type casting : Explicit Type Conversion By user

- ❑ Syntax: (typename) expression

TYPE CONVERSION AND TYPE CASTING

```
void main ()  
{  
int a = 5,b;  
float c = 5.5;  
double d = 4.0;  
b = a * c + d/10;  
printf("Ans is = %d", b );  
getch();  
}
```

Output:

Ans is = 27

TYPE CONVERSION AND TYPE CASTING

```
void main ()  
{  
int sum = 47,n= 10;  
float avg;  
avg = sum / n;  
printf("Ans is = %f", avg );  
avg = (float) sum / n;  
printf("Ans is = %f", avg );  
avg = (float) (sum / n);  
printf("Ans is = %f", avg );  
getch();  
}
```

Output: Ans is = 4.0

Ans is = 4.7

Ans is = 4.0

C INPUT OUTPUT (I/O)

□ `printf()` - The function sends formatted output to the screen.

- Here are some common format specifiers used in `printf()` :

1. `%d` - Integer
2. `%f` - Floating-point number
3. `%c` - Character
4. `%s` - String

C INPUT OUTPUT (I/O)

printf() example1

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
    int testInteger = 5;
```

```
    printf("Number = %d", testInteger);
```

```
}
```

C INPUT OUTPUT (I/O)

printf() example2

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
float number1 = 13.5;
```

```
double number2 = 12.4;
```

```
    printf("number1 = %f\n", number1);
```

```
    printf("number2 = %lf", number2);
```

```
}
```


C INPUT OUTPUT (I/O)

printf() example3

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
char chr = 'a';
```

```
    printf("character = %c", chr);
```

```
}
```

C INPUT OUTPUT (I/O)

□ **scanf()** - is used to read character, string, numeric data from keyboard.

- **Syntax of scanf**

- `scanf("%X", &variable);`

- where %X is the format specifier which tells the compiler what type of data is in a variable.

- & refers to address of “variable” which is directing the input value to a address returned by &variable.

C INPUT OUTPUT (I/O)

❑ **scanf()** - is used to read character, string, numeric data from keyboard.

Format specifier	Supported data types	Example	Description
%d	Integer	<u>scanf("%d", &a)</u>	Accept integer value such as 1, 5, 25, 105 <u>etc</u>
%f	Float	<u>scanf("%f", &b)</u>	Accept floating value such as 1.5, 15.20 <u>etc</u>
%c	Character	<u>scanf("%c", &c)</u>	Accept character value such as a, f, j, W, Z <u>etc</u>
%s	String	<u>scanf("%s", &d)</u>	Accept string value such as diet, <u>india etc</u>

C INPUT OUTPUT (I/O)

scanf() example1

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
    int testInteger;
```

```
    printf("Enter an integer: ");
```

```
    scanf("%d", &testInteger);
```

```
    printf("Number = %d",testInteger);
```

```
}
```


C INPUT OUTPUT (I/O)

scanf() example2

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
float num1;
```

```
double num2;
```

```
printf("Enter a number: ");
```

```
scanf("%f", &num1);
```

```
printf("Enter another number: ");
```

```
scanf("%lf", &num2);
```

```
printf("num1 = %f\n", num1);
```

```
printf("num2 = %lf", num2);
```

```
}
```

C INPUT OUTPUT (I/O)

scanf() example3

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
char chr;
```

```
    printf("Enter a character: ");
```

```
    scanf("%c",&chr);
```

```
    printf("You entered %c.", chr);
```

```
}
```


C INPUT OUTPUT (I/O)

scanf() example4

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
char chr;
```

```
printf("Enter a character: ");
```

```
scanf("%c", &chr);
```

```
printf("You entered %c.\n",chr);
```

```
printf("ASCII value is %d.", chr);
```

```
}
```

ASCII Value

C INPUT OUTPUT (I/O)

scanf() example4

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
int a;
```

```
float b;
```

```
printf("Enter integer and then a float: ");
```

```
scanf("%d %f", &a, &b);
```

```
printf("You entered %d and %f", a, b);
```

```
}
```


C INPUT OUTPUT (I/O)

□ getchar() and putchar()

- getchar function reads a single character from terminal.
- putchar function displays the character passed to it on the screen.

```
#include <stdio.h>
void main( )
{
    int c;
    printf("Enter a character: ");
    /* Take a character as input */
    c = getchar();
    /* Display the character */
    printf("Entered character is: ");
    putchar(c);
}
```

```
Enter a character: a
Entered character is: a
```

C INPUT OUTPUT (I/O)

□ gets() and puts ()

- gets function reads a line from stdin into the buffer pointed to by s until either a terminating newline or EOF (End of File) occurs.
- puts function writes the string 's' and 'a' trailing newline to stdout.

C INPUT OUTPUT (I/O)

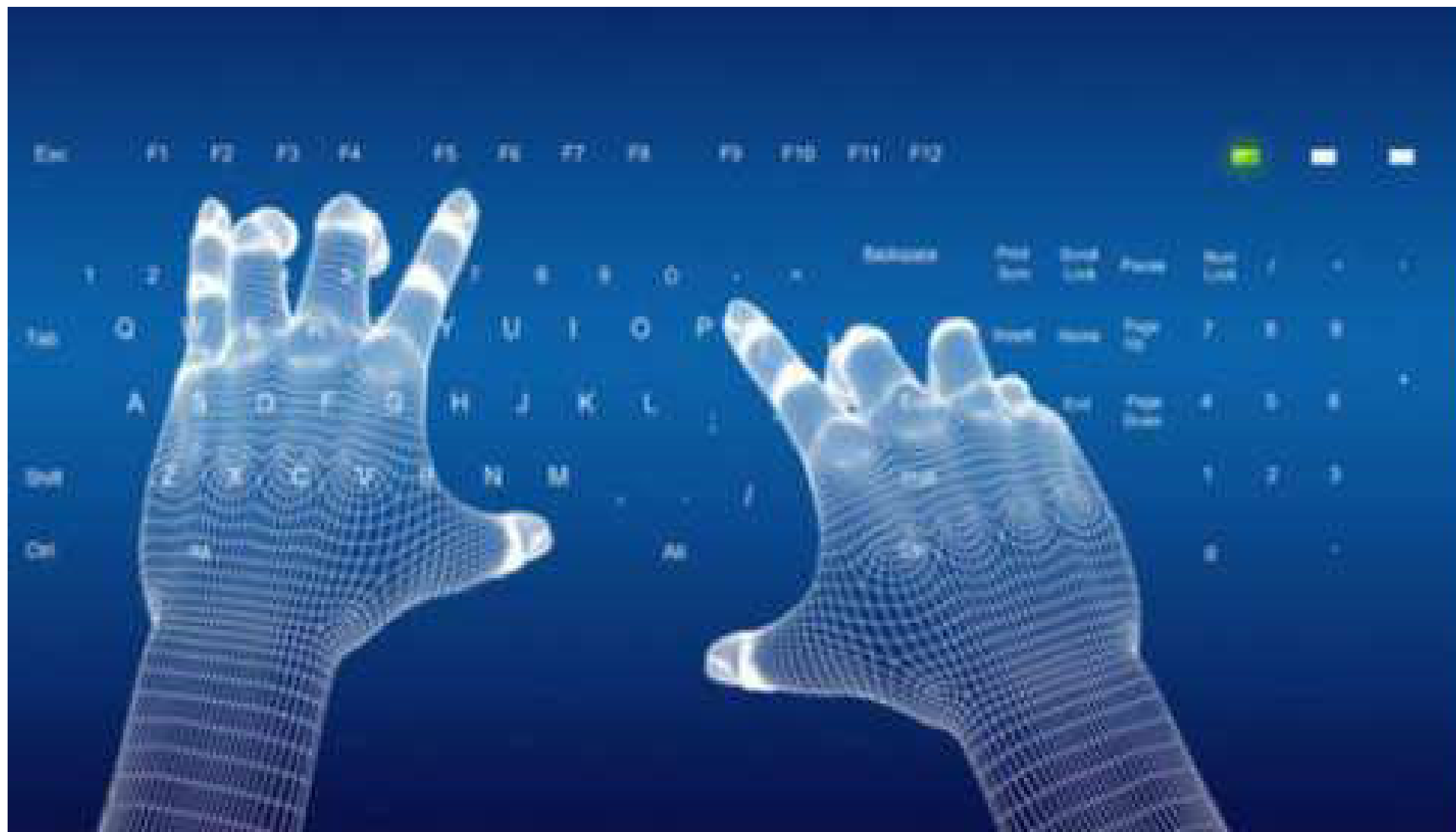
□ gets() and puts ()

```
#include <stdio.h>
void main( )
{
    /*Character array of length 100*/
    char str[100];
    printf("Enter a string: ");
    /* Take a string as input */

    gets( str );
    /* Display the string */

    printf("Entered string is: ");
    puts( str );
}
```

Enter a string: india
Entered string is: india



Thank You !